BLOCKCHAIN-BASED LAND REGISTRY SYSTEM

A PROJECT REPORT

Submitted by

Team BlockCodeM

Aditya Dhabhai [RA2512055010013]

Ishan Roy Barman [RA2512055010018]

Under the Guidance of

Kavi Priya D

Assistant Professor, Department of Computing Technologies

in partial fulfilment of the requirements for the course

Cloud Computing and Blockchain

M.Tech Computer Science and Engineering



DEPARTMENT OF COMPUTING TECHNOLOGIES

COLLEGE OF ENGINEERING AND TECHNOLOGY

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

KATTANKULATHUR – 603203

NOVEMBER 2026

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR – 603203

BONAFIDE CERTIFICATE

Certified that this Cloud Computing and Blockchain mini project report titled "**BLOCKCHAIN-BASED LAND REGISTRY SYSTEM**" is the bonafide work of "**Aditya Dhabhai [RA2512055010013]** and **Ishan Roy Barman [RA2512055010018]**" (Team BlockCodeM) who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

FACULTY
SIGNATURE
Kavi Priya D
Assistant Professor
Department of Computing Technologies
SRM Institute of Science and Technology,
Kattankulathur - 603203

HOD
SIGNATURE
Dr Niranjana G
Head of Department
Department of Computing Technologies
SRM Institute of Science and Technology,
Kattankulathur - 603203

TABLE OF CONTENTS

Chapter	Chapter Title	Sub-sections	Page No.
1	Introduction		1
		1.1 Overview of the Project	1
		1.2 Objectives of the Project	1
		1.3 Problem Definition	1
		1.4 Scope of the Application	2
		1.5 Methodology	2
		1.6 Technologies and Tools Used	3
2	Literature Survey and System Analysis		4
		2.1 Existing System	4
		2.2 Literature Survey	4
		2.3 Limitations of the Existing System	4
		2.4 Proposed System	5
		2.5 Feasibility Study	5
3	System Design		6
		3.1 System Architecture	6
		3.2 Module Description	6
		3.3 Data Flow Design	6
		3.4 Smart Contract Design	7
		3.5 Off-chain Data and Storage Design	8
		3.6 User Interface Design	8
4	Implementation		9
		4.1 Smart Contract Implementation	9
		4.2 Contract Compilation and Deployment	9
		4.3 Backend REST API	10
		4.4 Blockchain Connectivity	11
		4.5 Cloud Storage Integration	11
		4.6 Authentication and Middleware	11
		4.7 Frontend Implementation	11
5	Testing and Validation		12
		5.1 Testing Objectives	12
		5.2 Types of Testing	12
		5.3 Test Cases	12

Chapter	Chapter Title	Sub-sections	Page No.
		5.4 Validation of Inputs	13
		5.5 Issues Found and Resolution Plan	13
6	Output and Results		14
		6.1 Application Screenshots	14
		6.2 Output Explanation	20
		6.3 Performance Evaluation	20
7	Conclusion and Future Enhancements		22
		7.1 Summary	22
		7.2 Key Achievements	22
		7.3 Limitations	22
		7.4 Future Enhancements	23
	References		24

LIST OF FIGURES

Figure No.	Figure Title	Page No.
3.1	System architecture of the Blockchain Land Registry	6
3.2	Data flow for land registration / transfer	7
3.3	Document verification (tamper detection) flow	7
3.4	Smart contract data model (LandRegistry.sol)	8
4.1	Local Ethereum node log: deployment, registration and transfer transactions	10
6.1	Home page (public Search / History tab)	14
6.2	Registrar login page	14
6.3	Registrar signup page	15
6.4	Successful registrar signup	15
6.5	Register New Land form with sale-deed PDF	16
6.6	Land registered on blockchain: document hash and transaction hash	16
6.7	Duplicate Land ID rejected	17
6.8	Ownership timeline after registration (1 record)	17
6.9	Transfer Ownership form	18
6.10	Ownership transferred to Priya Sharma	18
6.11	Ownership timeline after transfer (2 records)	19
6.12	Genuine document: DOCUMENT VERIFIED	19
6.13	Edited document: TAMPERED / INVALID DOCUMENT	20
6.14	Search for an unregistered Land ID	20
6.15	API latency and on-chain gas consumption	21
7.1	Proposed AWS deployment architecture	23

LIST OF TABLES

Table No.	Table Title	Page No.
1.1	Technology stack	3
2.1	Summary of literature survey	4
2.2	Feasibility analysis	5
3.1	Module description	6
3.2	On-chain vs off-chain data placement	8
4.1	REST API endpoints	10
5.1	Functional and negative test cases	12
5.2	Issues identified during testing	13
6.1	API response time (n = 30 per operation)	21
6.2	Blockchain cost and accuracy metrics	21

ABSTRACT

Land records kept in centralised databases and paper registers can be altered by insiders, forged, or lost, which leads to fraud and long ownership disputes. This project, **"Blockchain-Based Land Registry System"**, developed by team BlockCodeM, records land ownership and every transfer on an Ethereum blockchain so that the history of a parcel becomes tamper-evident and publicly verifiable.

The system uses a hybrid on-chain / off-chain design. Each title document (PDF sale deed) is fingerprinted with SHA-256 and only the hash, together with the Land ID, owner, location and block timestamp, is stored in a **Solidity** smart contract deployed with **Hardhat** on a **Ganache** Ethereum network. The full document is stored in cloud object storage (**Cloudinary**, moving to **Amazon S3** in the final AWS deployment). A **Node.js / Express** REST API signs blockchain transactions through **ethers.js**, protects write operations with **JWT** authentication and bcrypt-hashed passwords, and accepts only PDF uploads. A **React + Vite** frontend provides four functions: register land, transfer ownership, search the complete ownership timeline and verify a document.

Verification recomputes the hash of an uploaded file and compares it with the on-chain hash inside the contract. In testing, all 30 genuine documents were verified and all 30 edited documents were detected as tampered. Read operations completed in about 15 ms, write transactions in about 115 ms on the local chain, and a registration consumed about 0.37 million gas. The project shows that blockchain combined with cloud storage gives a low-cost, transparent and reliable land record system.

Keywords: Blockchain, Ethereum, Smart Contract, Solidity, Land Registry, SHA-256, Cloud Storage, AWS, Tamper Detection.

CHAPTER 1: INTRODUCTION

1.1 Overview of the Project

Land is one of the most valuable assets a citizen owns, yet the records that prove ownership are still largely maintained in centralised databases and paper registers at sub-registrar offices. Such records can be altered by an insider, lost in a disaster, or forged, and each of these leads to costly and long-running ownership disputes. The **Blockchain Land Registry** developed by team **BlockCodeM** is a web application that records land ownership and every subsequent transfer on an Ethereum blockchain, while the actual title documents are kept in cloud storage.

The system follows the industry-standard **hybrid on-chain / off-chain pattern**. Each uploaded sale deed (PDF) is fingerprinted with the SHA-256 algorithm; only this 64-character hash, together with the Land ID, owner name, location and a block timestamp, is written to the LandRegistry smart contract. The full document is stored in cloud object storage. Anyone can later upload a document and the system recomputes its hash and compares it with the hash on the chain, so a single changed byte in a deed is detected immediately.

The application has three layers: a **React + Vite** single-page frontend, a **Node.js / Express** REST backend that handles authentication, file upload, hashing and blockchain interaction through **ethers.js**, and a **Solidity** smart contract deployed on a local Ethereum network (Ganache / Hardhat Network). Documents are uploaded to **Cloudinary** cloud storage, and the final phase moves hosting and storage to **Amazon Web Services (AWS)**.

1.2 Objectives of the Project

1. **Tamper-proof ownership records:** store land records on an immutable blockchain ledger so that no single party can silently change them.
2. **Complete ownership history:** maintain an append-only timeline of every owner of a parcel with timestamps.
3. **Document authenticity verification:** allow any citizen, bank or court to check whether a deed is genuine by comparing its SHA-256 hash with the on-chain hash.
4. **Secure cloud storage of documents:** keep large PDF documents off-chain in cloud storage to save gas cost while keeping them retrievable.
5. **Controlled write access:** only authenticated registrars (JWT login) may register or transfer land; search and verification stay public.
6. **Hands-on exposure** to blockchain (Ethereum, Solidity, Hardhat, Ganache) and cloud services (Cloudinary, AWS) as required by the course.

1.3 Problem Definition

Land records managed through centralised systems face the following problems:

- **Unauthorised modification:** a database administrator or a compromised account can edit ownership data without leaving a reliable trace.
- **Forged documents:** fake or edited sale deeds are hard to detect visually, which enables fraud such as selling the same land twice.
- **Opaque history:** previous owners and the chain of title are spread across physical registers and are slow to trace.
- **Single point of failure:** a central server or paper archive can be lost, damaged or made unavailable.
- **Slow verification:** banks and buyers must visit offices and wait days to confirm a title.

Problem statement: design and implement a blockchain-based system that records land ownership and transaction details securely, stores supporting documents off-chain in the cloud, and provides a transparent, verifiable and tamper-evident ownership history.

1.4 Scope of the Application

Functional scope

- Registrar sign-up and login with bcrypt-hashed passwords and 24-hour JWT tokens.
- Register a new land parcel (Land ID, owner, location and sale-deed PDF) with duplicate-ID protection.
- Transfer ownership with a new deed while the previous record stays in the history.
- Search any Land ID and view the complete ownership timeline from the blockchain.
- Verify any document against the current on-chain hash (VERIFIED / TAMPERED).

Out of scope (current version)

- Payment of stamp duty or sale consideration on-chain.
- Public mainnet deployment and citizen wallets (planned as Sepolia + MetaMask).
- Survey maps / GIS boundary data.

1.5 Methodology

The project followed an **iterative development model** aligned with the three course reviews:

Phase	Activities	Review
1. Problem identification	Study of land-record issues, literature survey of blockchain land registries	Review 1 (17 Aug 2026)
2. Design	Architecture, smart contract data model, API design, UI wireframes	Review 2 (17 Sep 2026)
3. Implementation	Solidity contract, Hardhat deployment, Express API, React UI, Cloudinary upload, JWT auth	Review 2 / 3
4. Testing & evaluation	Functional and negative test cases, latency and gas measurements	Review 3 (27 Oct 2026)

Phase	Activities	Review
5. Cloud deployment & documentation	AWS migration, final report and presentation	Final Review (11 Nov 2026)

1.6 Technologies and Tools Used

Table 1.1: Technology stack

Layer	Technology	Purpose
Smart contract	Solidity ^0.8.19	LandRegistry contract (records, history, verification)
Blockchain network	Ganache / Hardhat Network	Local Ethereum chain for development and demo
Framework	Hardhat 2 + hardhat-toolbox	Compile and deploy contracts
Chain library	ethers.js v6	JSON-RPC provider, wallet signing, contract calls
Backend	Node.js 22, Express 5	REST API server
File upload	Multer 2	Multipart parsing, PDF-only filter, 10 MB limit
Hashing	Node.js crypto (SHA-256)	Document fingerprint
Authentication	bcryptjs, jsonwebtoken	Password hashing, JWT tokens
Cloud storage	Cloudinary (raw upload)	Off-chain storage of PDF deeds
Cloud hosting (final)	AWS S3, EC2, CloudWatch	Hosting, document storage and monitoring
Frontend	React 19, Vite 8, Axios	Single-page user interface
Tools	VS Code, Git/GitHub, Remix IDE, MetaMask	Development, version control, contract testing

CHAPTER 2: LITERATURE SURVEY AND SYSTEM ANALYSIS

2.1 Existing System

In India, land records are maintained by state revenue departments and registered at sub-registrar offices. Programmes such as the **Digital India Land Records Modernization Programme (DILRMP)** and state portals such as Karnataka's **Bhoomi** have digitised records, but the data still lives in centralised government databases. Ownership transfer requires the physical presence of the parties, stamp-paper deeds and manual entry by officials. Verification by a buyer or a bank means requesting an encumbrance certificate and waiting for it to be issued.

2.2 Literature Survey

Table 2.1: Summary of literature survey

#	Work	Key idea	Relevance to this project
1	Nakamoto (2008) [1]	Bitcoin: hash-linked blocks and proof-of-work give an append-only public ledger	Basis of immutability of our records
2	Buterin (2014) [2], Wood (2014) [3]	Ethereum adds a Turing-complete virtual machine and smart contracts	Our registry logic runs as a Solidity contract
3	Lantmäteriet testbed, Sweden (2017) [7]	National land authority pilot of property transfer on a blockchain	Shows real-world value of on-chain transfer history
4	Thakur et al. (2020) [4]	Framework and challenges for blockchain land titling in India	Indian context; motivates hash-on-chain design
5	Krishnapriya & Sarath (2020) [5]	Ethereum smart-contract based land registration	Similar register / transfer workflow
6	Shuaib et al. (2020) [6]	Secure and reliable blockchain land registry framework	Role separation and security considerations
7	Benet (2014) [9]	IPFS content-addressed off-chain storage	Supports off-chain document storage with hash links

The survey shows that storing whole documents on a blockchain is expensive, and that the common practice is to keep a cryptographic hash on-chain and the file off-chain. It also shows that most academic prototypes stop at the smart contract and do not offer a complete web workflow with cloud storage and tamper verification. This project addresses that gap.

2.3 Limitations of the Existing System

- Records can be changed by privileged insiders without an independent audit trail.
- Forged or edited deeds cannot be detected without manual expert checks.
- History of ownership is fragmented and slow to obtain.
- Centralised storage is a single point of failure.

- Citizens cannot verify a record themselves in real time.

2.4 Proposed System

The proposed system stores each land record as a LandRecord structure in an Ethereum smart contract. Every registration or transfer is a signed transaction that is mined into a block, and the contract appends the new record to the parcel's history array, so earlier owners can never be removed. Supporting PDFs are uploaded to cloud storage and only their SHA-256 hashes are stored on-chain. A public verification endpoint lets anyone check a document in a few milliseconds.

Features of the proposed system

- Immutable, timestamped ownership timeline for every Land ID.
- Instant tamper detection by hash comparison inside the contract (verifyHash).
- Low on-chain cost: about 0.37 million gas per registration (Section 6.3).
- Role separation: registrars write (JWT protected), public users read and verify.
- Cloud document storage with secure HTTPS URLs.

2.5 Feasibility Study

Table 2.2: Feasibility analysis

Type	Assessment
Technical	All components are open-source and mature (Solidity, Hardhat, ethers.js, Express, React). The system runs on a normal laptop with Node.js 18+.
Economic	Development cost is zero (local chain, free Cloudinary / AWS free-tier). On a public chain, only small hashes are stored, which keeps gas cost low.
Operational	Registrars use a simple four-tab web UI; no blockchain knowledge is needed because the backend signs transactions.
Legal / social	Supports, rather than replaces, the legal registration process by adding an independent tamper-evident audit trail.

CHAPTER 3: SYSTEM DESIGN

3.1 System Architecture

Figure 3.1 shows the four-tier architecture. The React client talks only to the Express API. The API is the single component that holds the blockchain wallet key and the cloud storage credentials, so neither is exposed to the browser.

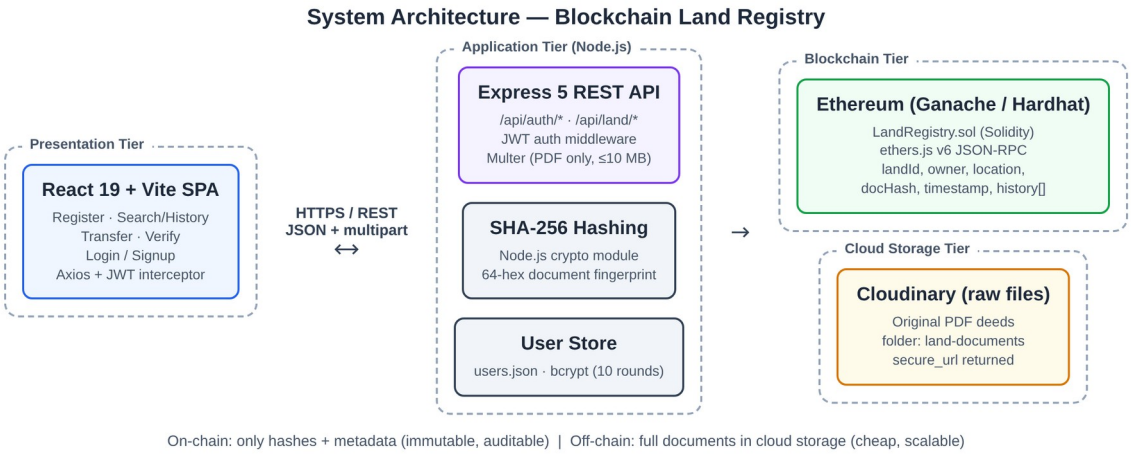


Figure 3.1: System architecture of the Blockchain Land Registry

3.2 Module Description

Table 3.1: Module description

Module	File(s)	Responsibility
Smart contract	blockchain/contracts/LandRegistry.sol	Stores records and history, enforces no-duplicate and must-exist rules, verifies hashes
Deployment	blockchain/scripts/deploy.js, hardhat.config.cjs	Compiles and deploys contract to Ganache
Chain connector	backend/contract.js	ethers JsonRpcProvider + Wallet + Contract instance
REST API	backend/server.js	Register, transfer, history and verify endpoints; hashing; error handling
Authentication	backend/auth.js	Signup / login, bcrypt, JWT issue and authMiddleware
Cloud storage	backend/cloudinary.js	uploadToCloud() returns secure URL of stored PDF
User interface	frontend/src/App.jsx, components/*.jsx	Tabs: Register, Search/History, Transfer, Verify, Login, Signup

3.3 Data Flow Design

Figure 3.2 shows the step-by-step data flow when a registrar registers a parcel. Ownership transfer follows the same path but calls transferOwnership() and requires the Land ID to exist.

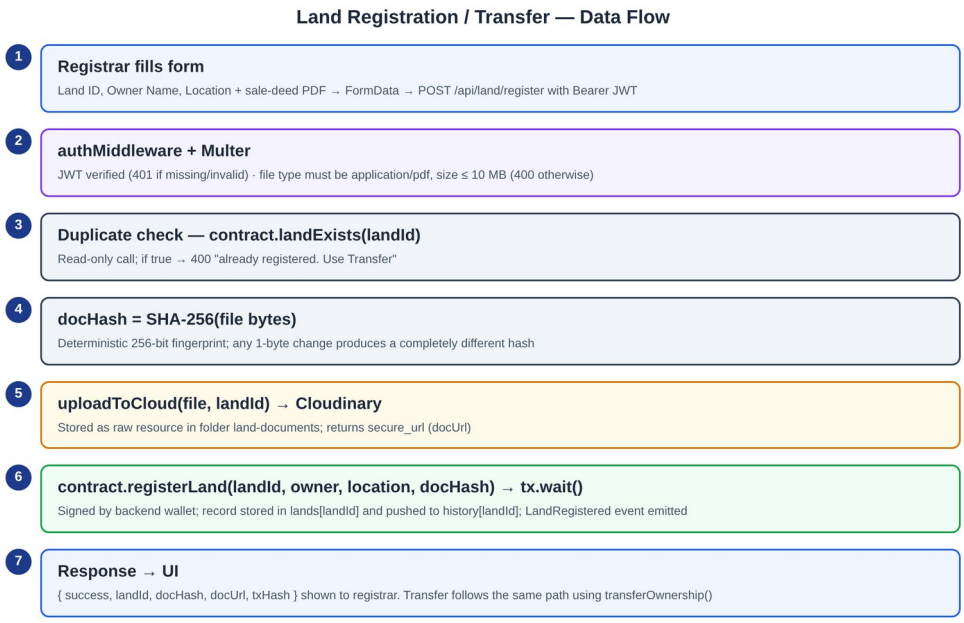


Figure 3.2: Data flow for land registration / transfer

Figure 3.3 shows the verification flow. Verification is a read-only (view) call, so it costs no gas and does not need a login.

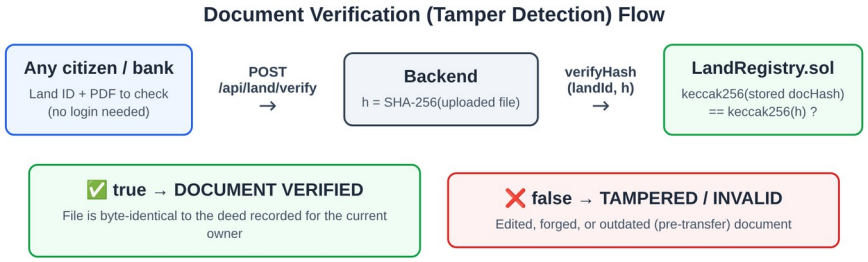


Figure 3.3: Document verification (tamper detection) flow

3.4 Smart Contract Design

The contract keeps two mappings keyed by Land ID: lands holds the current record for O(1) lookup, and history holds an append-only array of all records. Events are emitted for every write so that external tools can index the activity (Figure 3.4).

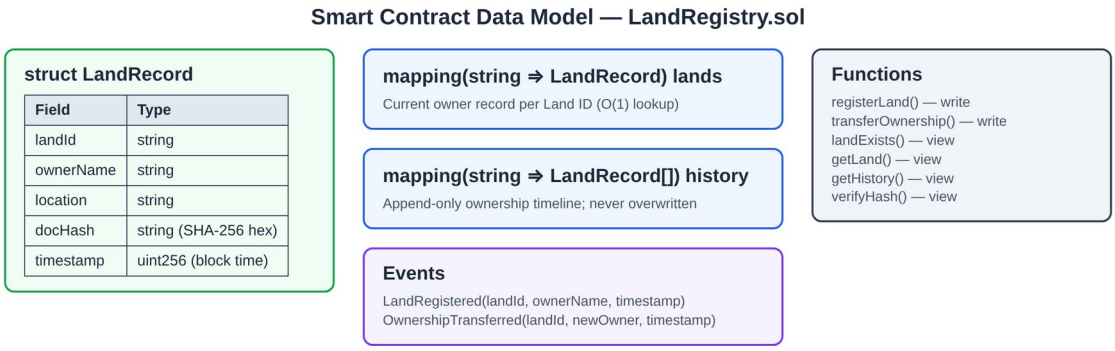


Figure 3.4: Smart contract data model (LandRegistry.sol)

3.5 Off-chain Data and Storage Design

Table 3.2: On-chain vs off-chain data placement

Data	Where stored	Reason
Land ID, owner, location, docHash, timestamp	Ethereum contract storage	Must be immutable and publicly verifiable
Full PDF deed	Cloudinary (folder land-documents) → AWS S3 in final phase	Large binary; too costly on-chain
Registrar accounts (name, email, bcrypt hash)	backend/users.json → AWS RDS/DynamoDB in final phase	Private data; must not be public on-chain
JWT secret, wallet key, cloud keys	backend/.env (not committed)	Secrets kept server-side only

3.6 User Interface Design

The UI is a single page with a dark theme and four tabs. Register and Transfer show a lock icon until the registrar logs in; Search/History and Verify are always public. Each form shows a loading label while the blockchain transaction is being mined, and results (hash, transaction hash, timeline or verification verdict) are displayed in a highlighted card. Screenshots of every screen are given in Chapter 6.

CHAPTER 4: IMPLEMENTATION

4.1 Smart Contract Implementation

The core write function registerLand() rejects duplicates, builds a LandRecord with the block timestamp, stores it and appends it to the history. transferOwnership() keeps the location, replaces the owner and document hash, and appends the new record.

```
function registerLand(string memory landId, string memory ownerName,
                      string memory location, string memory docHash) public {
    require(!landExists(landId), "Land already registered");
    LandRecord memory record = LandRecord({ landId: landId, ownerName:
ownerName,
        location: location, docHash: docHash, timestamp: block.timestamp });
    lands[landId] = record;
    history[landId].push(record);          // first entry of the timeline
    emit LandRegistered(landId, ownerName, block.timestamp);
}

function verifyHash(string memory landId, string memory hash) public view
returns (bool) {
    require(landExists(landId), "Land not registered");
    return keccak256(abi.encodePacked(lands[landId].docHash)) ==
           keccak256(abi.encodePacked(hash));
}
```

Solidity cannot compare strings directly, so verifyHash() compares the keccak256 digests of the stored and supplied hash strings.

4.2 Contract Compilation and Deployment

The contract is compiled with Hardhat and deployed to the local Ethereum network with scripts/deploy.js. The deployed address is placed in backend/.env. Figure 4.1 shows the real node log from the demo run: the deployment transaction (1,425,762 gas), the registration of LAND101 and its transfer, each mined in its own block.

```
$ npx hardhat node --port 8545
Started HTTP and WebSocket JSON-RPC server at http://127.0.0.1:8545/

$ node deploy (LandRegistry.json bytecode)
LandRegistry deployed to: 0x5FbDB2315678afecb367f032d93F642f64180aa3

--- node log ---
eth_sendRawTransaction
  Contract deployment: <UnrecognizedContract>
  Contract address:    0x5fbdb2315678afecb367f032d93f642f64180aa3
  Transaction:         0x30e372e55825a56bea6a2add18715986980a03b7d9ec135710af6f47579c305c
  From:                0xf39fd6e51aad88f6f4ce6ab8827279cfffb92266
  Value:               0 ETH
  Gas used:            1425762 of 1425762
  Block #1:            0x9b78d9bef11111617115787bbc38a543610d7fba3372d5a57abead552f3596b6

# registerLand(LAND101)
eth_sendRawTransaction
  Contract call:       <UnrecognizedContract>
  Transaction:         0xf0855d1ae14829e27c72f2f05162559fadf2c5f15761bc29815bdf5b746f922a
  From:                0xf39fd6e51aad88f6f4ce6ab8827279cfffb92266
  To:                  0x5fbdb2315678afecb367f032d93f642f64180aa3
  Value:               0 ETH
  Gas used:            458781 of 458781
  Block #2:            0x5a1c097b540b59efe24388cbfe3bea4290be94565462f3995f1dcf264f1492e6

# transferOwnership(LAND101 -> Priya Sharma)
eth_sendRawTransaction
  Contract call:       <UnrecognizedContract>
  Transaction:         0xf417b419fc06ce45bfd8d3132567694313a10f44ebde40fcd7196cf53ee0e185
  From:                0xf39fd6e51aad88f6f4ce6ab8827279cfffb92266
  To:                  0x5fbdb2315678afecb367f032d93f642f64180aa3
  Value:               0 ETH
  Gas used:            273933 of 273933
  Block #3:            0x93848611c28e1064bcd15fe62f8b970ce77eb2c9379aeb49d153b08e06c6de48

$ cd backend && node server.js
Backend running on http://localhost:5001
```

Figure 4.1: Local Ethereum node log: deployment, registration and transfer transactions

4.3 Backend REST API

Table 4.1: REST API endpoints

Method	Endpoint	Auth	Description
POST	/api/auth/signup	Public	Create registrar account (bcrypt hash)
POST	/api/auth/login	Public	Returns JWT (24 h) and user name
POST	/api/land/register	JWT	Multipart: landId, ownerName, location, document
POST	/api/land/transfer	JWT	Multipart: landId, newOwner, document
GET	/api/land/:id/history	Public	Full ownership timeline
POST	/api/land/verify	Public	Multipart: landId, document → verified true/false

The document hash is computed on the server from the uploaded bytes, so a client cannot submit a fake hash:

```
function hashFile(path) {
  const fileBuffer = fs.readFileSync(path);
  return crypto.createHash("sha256").update(fileBuffer).digest("hex");
}
// inside POST /api/land/register
const docHash = hashFile(req.file.path);
const docUrl  = await uploadToCloud(req.file.path, landId);
const tx = await contract.registerLand(landId, ownerName, location, docHash);
await tx.wait();                                    // wait until mined
res.json({ success: true, landId, docHash, docUrl, txHash: tx.hash });
```

4.4 Blockchain Connectivity

backend/contract.js creates an ethers.js JsonRpcProvider for the Ganache URL, a Wallet from the server's private key and a Contract object from the compiled ABI. All write calls are therefore signed by the registry's server wallet, and read calls (landExists, getHistory, verifyHash) are free eth_call requests.

4.5 Cloud Storage Integration

Uploaded PDFs are sent to Cloudinary as raw resources in the land-documents folder with a public ID of the form LANDID_timestamp. The returned secure_url is sent back to the client together with the hash and transaction hash.

```
async function uploadToCloud(filePath, landId) {
  const result = await cloudinary.uploader.upload(filePath, {
    resource_type: "raw", folder: "land-documents",
    public_id: `${landId}_${Date.now()}`,
  });
  return result.secure_url;
}
```

4.6 Authentication and Middleware

Passwords are hashed with bcrypt (10 salt rounds). On login a JWT signed with JWT_SECRET is issued for 24 hours. authMiddleware checks the Bearer token on register and transfer routes and returns 401 for a missing or invalid token. Multer accepts only application/pdf files up to 10 MB, and a final Express error handler converts Multer errors into JSON responses.

4.7 Frontend Implementation

The React app (App.jsx) keeps the active tab, token and user name in state and in localStorage. A shared Axios instance (api.js) automatically adds the Authorization header to every request. Each feature is a separate component (RegistryLand, TransferLand, SearchLand, VerifyDoc, Login, Signup) that builds a FormData object, calls the API and renders the result or error.

CHAPTER 5: TESTING AND VALIDATION

5.1 Testing Objectives

- Confirm that every functional requirement (register, transfer, history, verify, auth) works end-to-end on a real chain.
- Confirm that invalid input and unauthorised requests are rejected with clear messages.
- Confirm that tampered or outdated documents are always detected.

5.2 Types of Testing

- **Functional (UI) testing:** complete demo flow driven in the browser with Playwright (Chapter 6 screenshots).
- **API testing:** each endpoint called with curl / fetch using valid and invalid inputs.
- **Smart contract testing:** require() rules checked through duplicate and unknown Land IDs.
- **Performance testing:** 30 iterations of every operation, plus 50 concurrent history requests.

5.3 Test Cases

Table 5.1: Functional and negative test cases

ID	Test case	Expected	Actual result	Status
TC01	Signup with new email	Account created	"Signup successful! Please login."	Pass
TC02	Signup with existing email	400 error	400 "Email already registered"	Pass
TC03	Signup with missing fields	400 error	400 "All fields required"	Pass
TC04	Login with wrong password	401 error	401 "Invalid email or password"	Pass
TC05	Register without token	401 error	401 "Login required"	Pass
TC06	Register with forged token	401 error	401 "Invalid or expired token"	Pass
TC07	Register LAND101 with PDF	Tx mined, hash shown	docHash + txHash returned (Fig. 6.6)	Pass
TC08	Register duplicate LAND101	Rejected	400 "already registered. Use Transfer"	Pass
TC09	Upload .txt instead of PDF	Rejected	400 "Only PDF files are allowed"	Pass
TC10	Transfer LAND101 to new owner	History grows to 2	2 records shown (Fig. 6.11)	Pass
TC11	Transfer unknown Land ID	Rejected	400 "Land NOPE is not registered yet."	Pass
TC12	Verify current deed	VERIFIED	verified: true (Fig. 6.12)	Pass
TC13	Verify edited deed	TAMPERED	verified: false (Fig. 6.13)	Pass
TC14	Verify old deed after transfer	Not valid	verified: false	Pass
TC15	History of unknown Land ID	Clear error	"Land not registered" (HTTP 500)	Partial
TC16	Register / verify without file	400 error	500 "Cannot read properties of undefined"	Fail

5.4 Validation of Inputs

- **Frontend:** every form checks that all fields and the file are present before calling the API.
- **File:** MIME type must be application/pdf; size at most 10 MB.
- **Business rules (contract):** a Land ID cannot be registered twice; transfer, history and verify require an existing Land ID.
- **Auth:** register and transfer require a valid, unexpired JWT.

5.5 Issues Found and Resolution Plan

Table 5.2: Issues identified during testing

Issue	Cause	Fix
Missing file gives HTTP 500 (TC16)	req.file is undefined and hashFile() is called without a check	Return 400 "Document is required" when req.file is missing
Unknown Land ID returns 500 (TC15)	Contract revert is passed through as a server error	Call landExists() first and return 404
"Nonce too low" when two writes are sent within about 250 ms	ethers.js provider caches the pending nonce	Wrap the wallet in ethers NonceManager or queue write requests
Uploaded temp files stay in uploads/	Multer temp files are not deleted after cloud upload	fs.unlink() after the hash is computed and the upload finishes

CHAPTER 6: OUTPUT AND RESULTS

6.1 Application Screenshots

The following screenshots were captured from the running application (React frontend, Express backend, contract deployed on the local Ethereum node) during the complete demo flow for parcel LAND101.

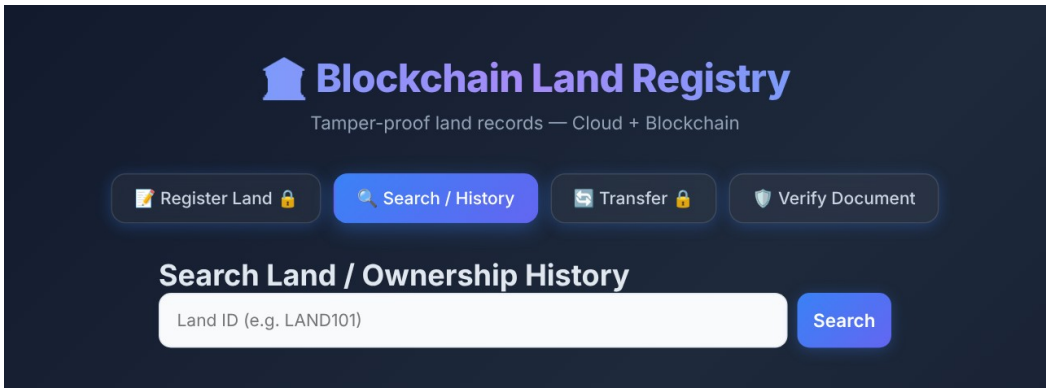


Figure 6.1: Home page (public Search / History tab)

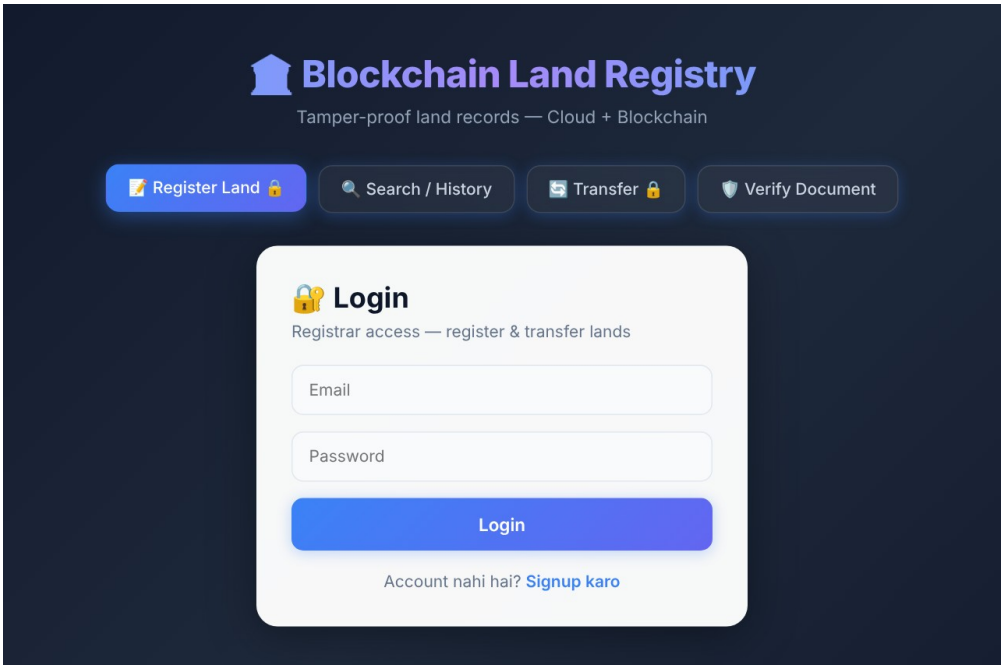


Figure 6.2: Registrar login page

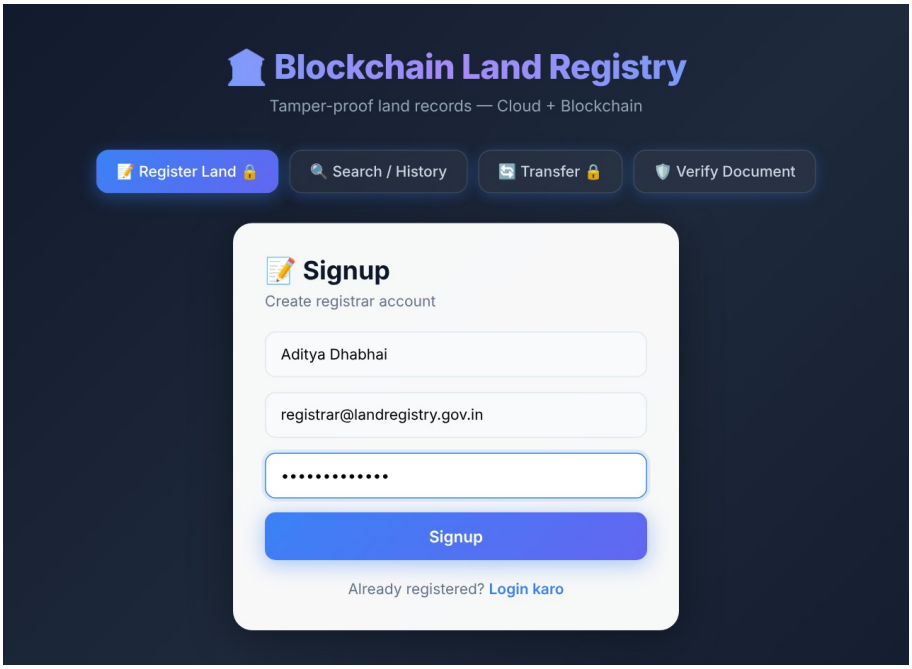


Figure 6.3: Registrar signup page

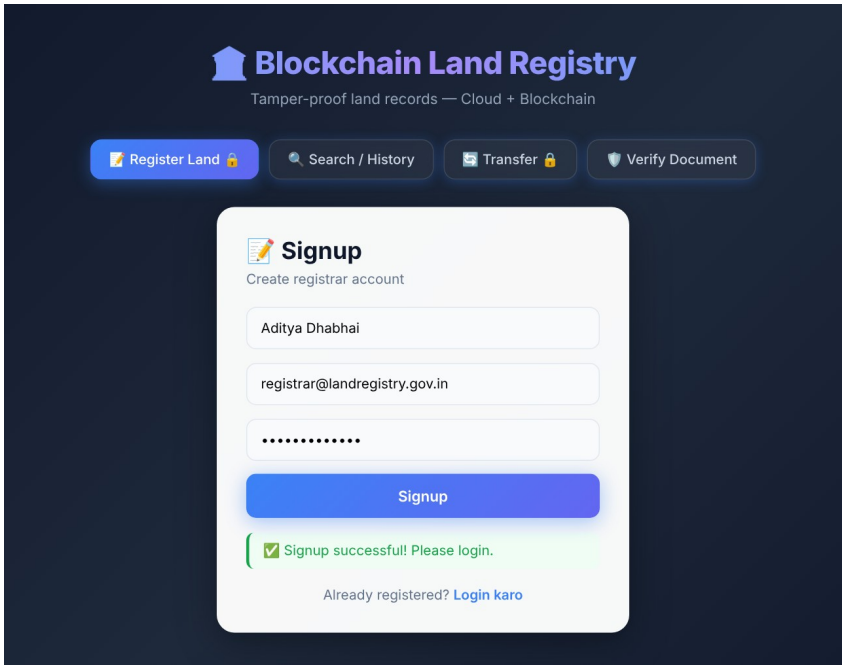


Figure 6.4: Successful registrar signup

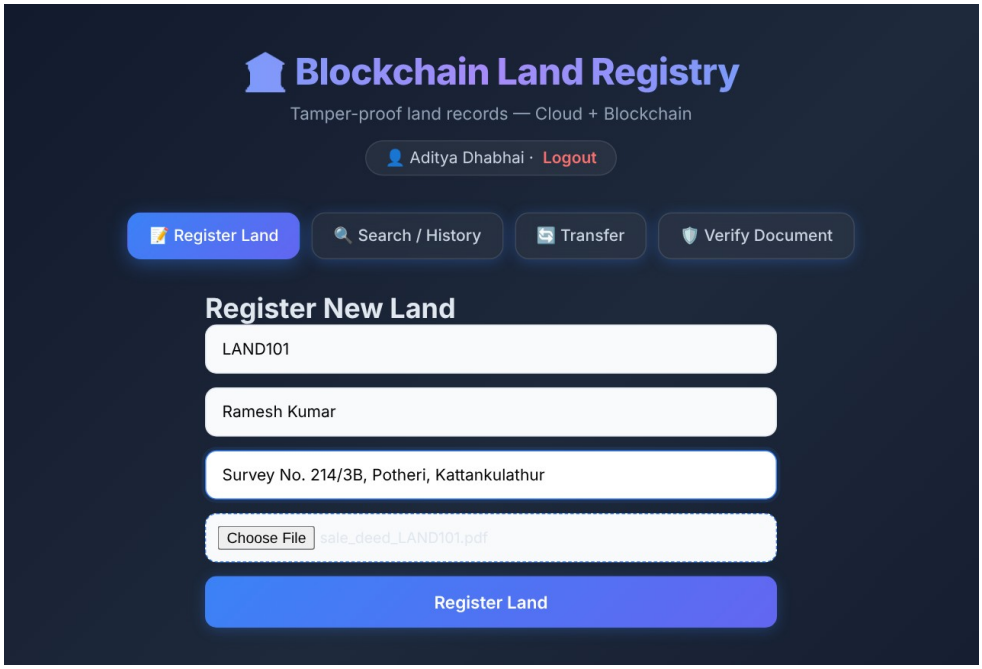


Figure 6.5: Register New Land form with sale-deed PDF

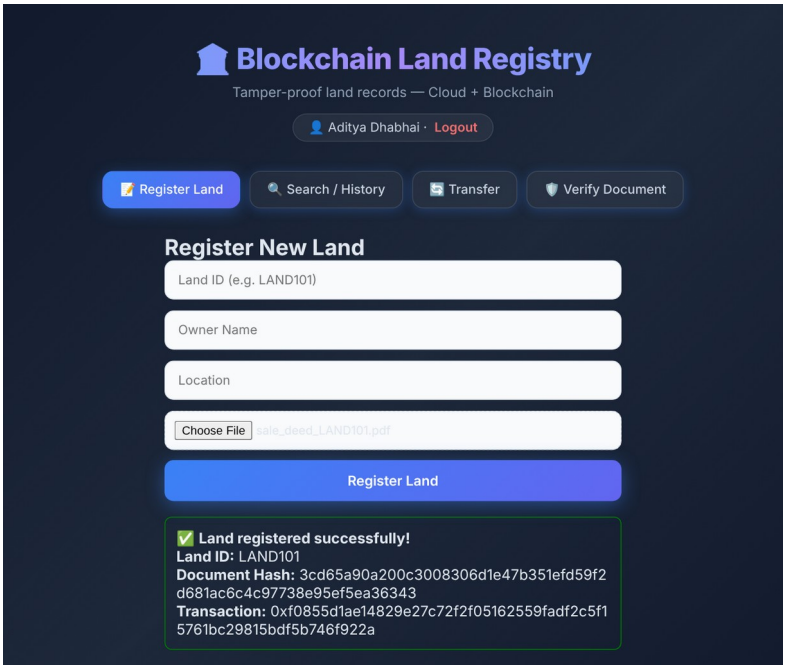


Figure 6.6: Land registered on blockchain: document hash and transaction hash

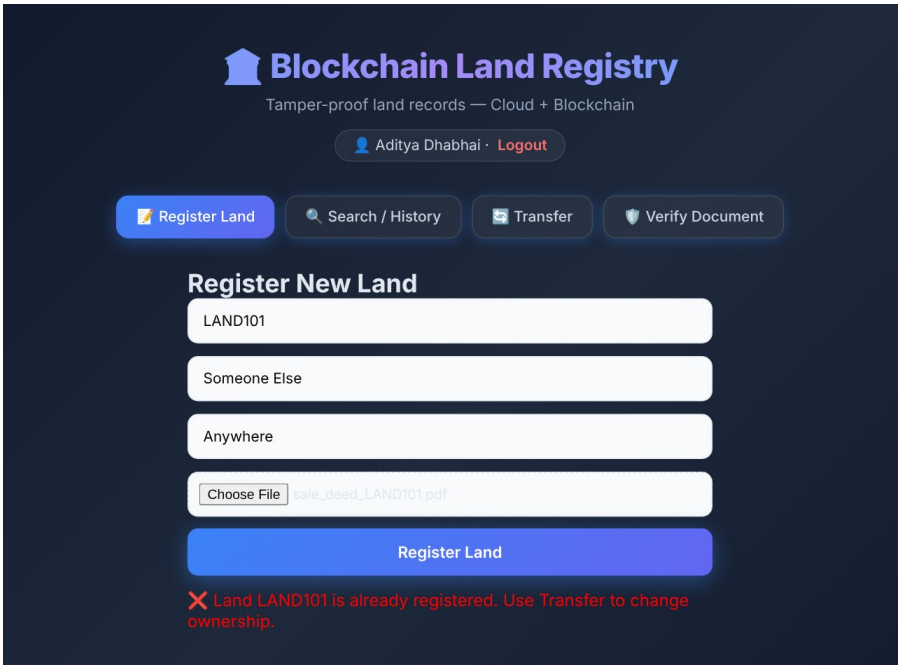


Figure 6.7: Duplicate Land ID rejected

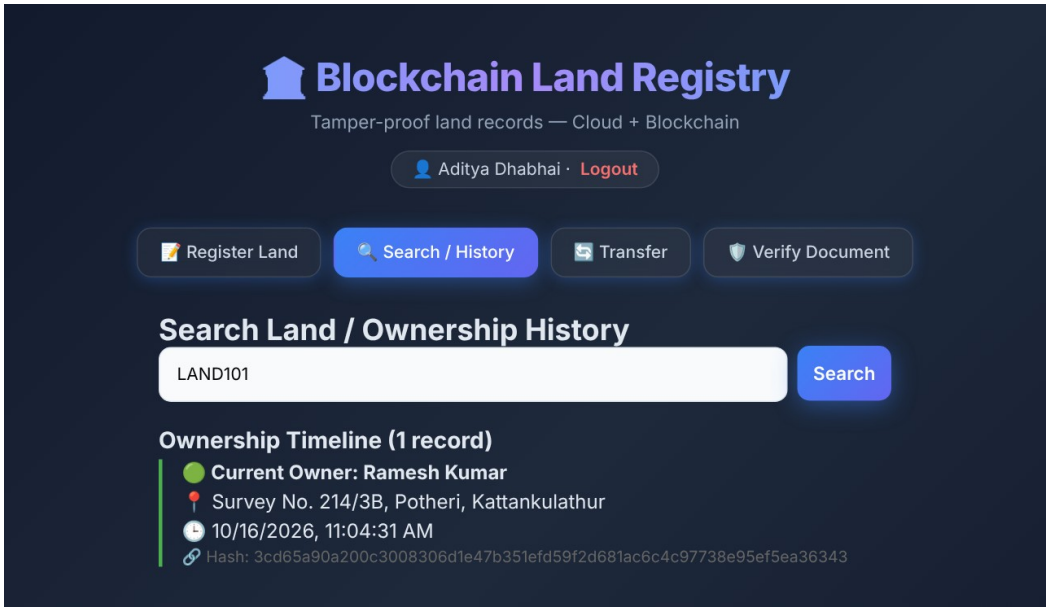


Figure 6.8: Ownership timeline after registration (1 record)

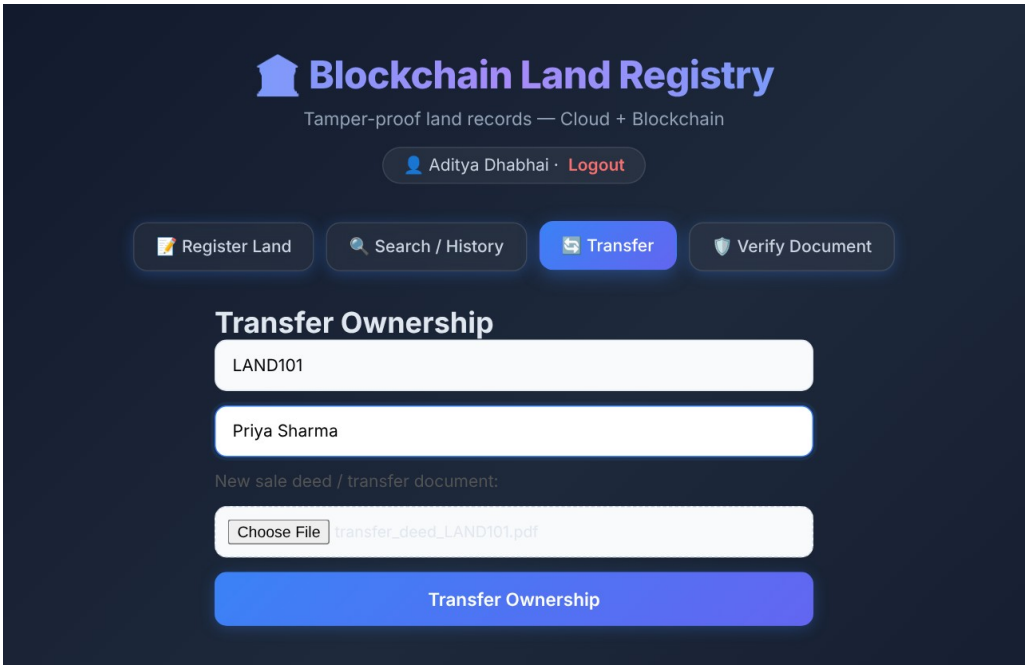


Figure 6.9: Transfer Ownership form

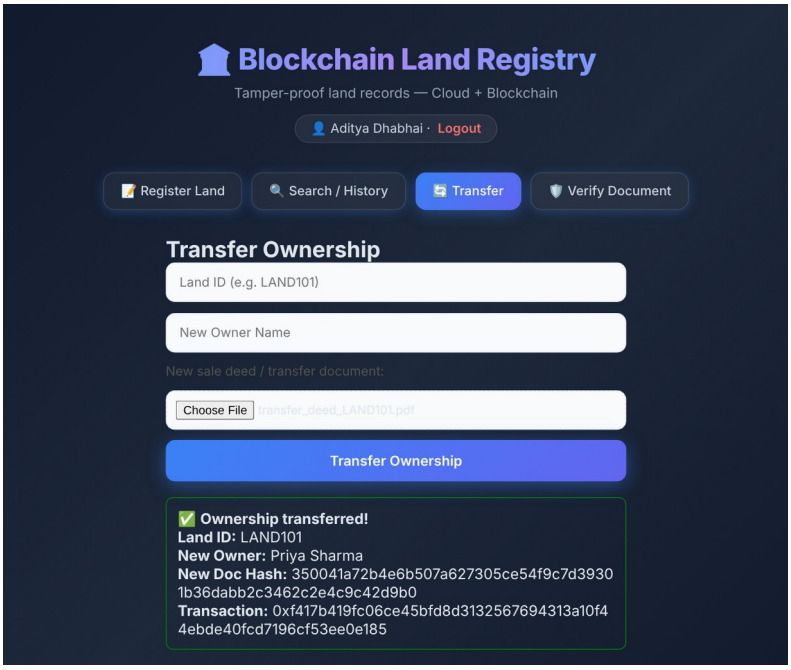


Figure 6.10: Ownership transferred to Priya Sharma

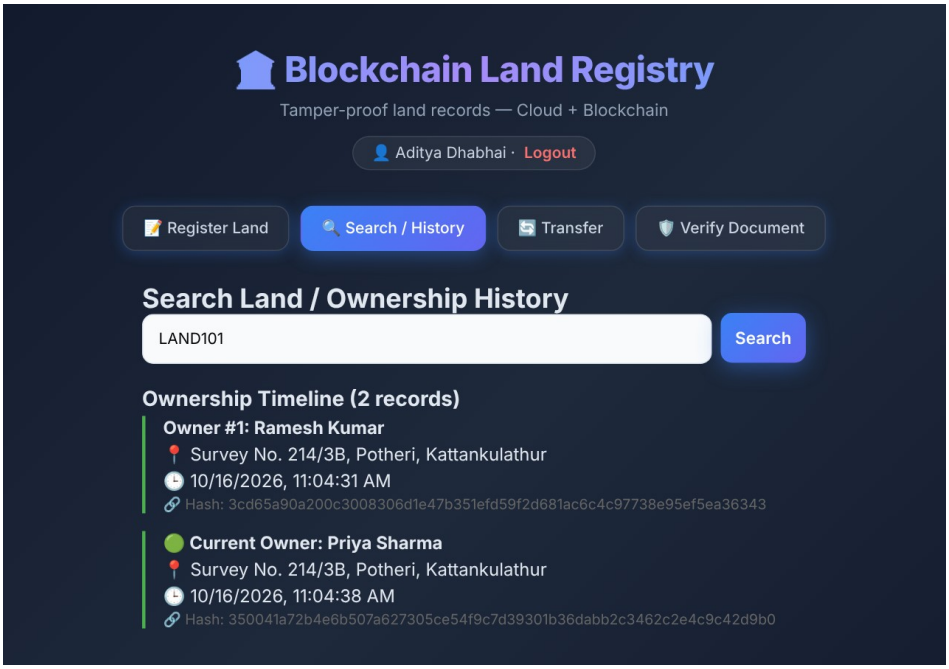


Figure 6.11: Ownership timeline after transfer (2 records)

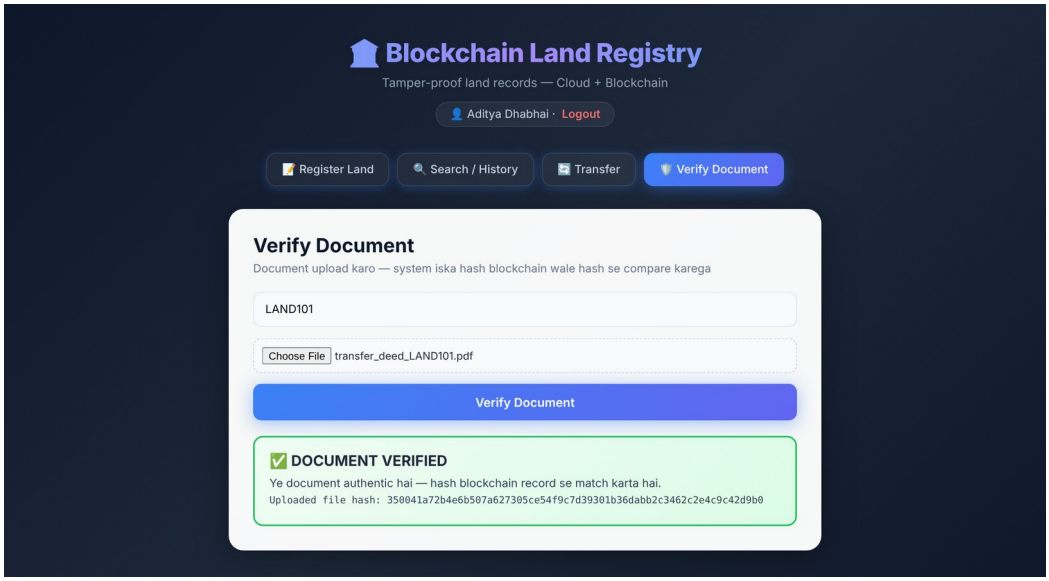


Figure 6.12: Genuine document: DOCUMENT VERIFIED

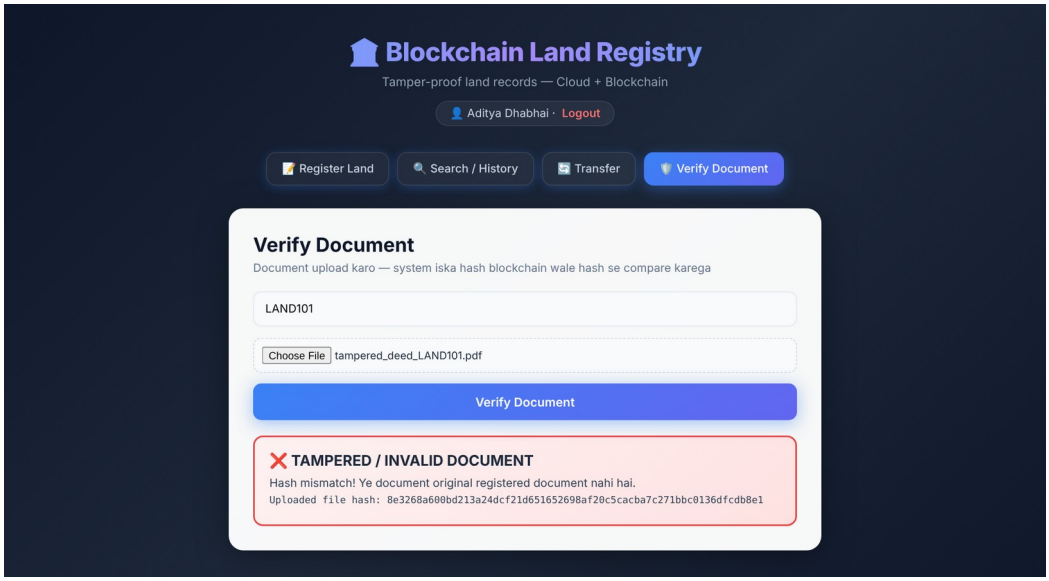


Figure 6.13: Edited document: TAMPERED / INVALID DOCUMENT

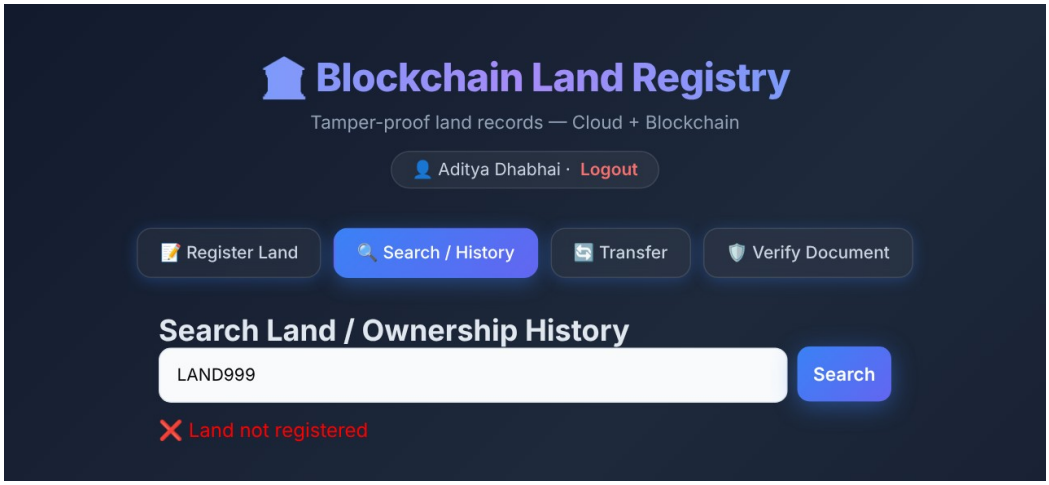


Figure 6.14: Search for an unregistered Land ID

6.2 Output Explanation

- **Registration (Fig. 6.6):** the API returns the 64-hex SHA-256 hash of the deed and the Ethereum transaction hash, proving that the record was mined.
- **History (Fig. 6.8, 6.11):** after the transfer the earlier owner, Ramesh Kumar, is still listed as Owner #1 with the original deed hash, and Priya Sharma is the current owner. Nothing is overwritten.
- **Verification (Fig. 6.12, 6.13):** the current transfer deed matches the chain. A copy whose buyer name and amount were edited produces a completely different hash and is flagged as tampered.

6.3 Performance Evaluation

Each operation was executed 30 times through the REST API on the local deployment (Hardhat Network with automine, Node.js 22). Write latency includes hashing, the contract transaction

and waiting for the receipt. Cloud upload time is not included, because it depends on the network link to the cloud provider.

Table 6.1: API response time (n = 30 per operation)

Operation	Avg (ms)	Min (ms)	Max (ms)	P95 (ms)
Register land (write tx)	117.1	111.1	129.4	121.1
Transfer ownership (write tx)	114.4	107.8	123.4	122.4
Get history (view call)	15.2	13.7	18.3	17.1
Verify genuine document	15.7	13.4	20.3	19.1
Verify tampered document	15.4	14.1	17.1	16.3

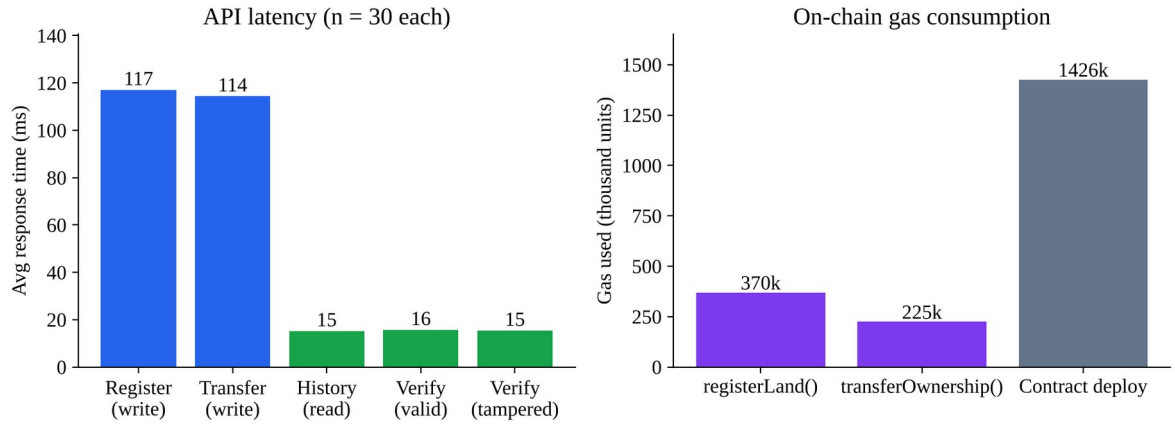


Figure 6.15: API latency and on-chain gas consumption

Table 6.2: Blockchain cost and accuracy metrics

Metric	Measured value	Remark
Contract deployment gas	1,425,762	One-time cost
registerLand() gas	369,571 (avg)	Grows with string length; LAND101 with long address used 458,781
transferOwnership() gas	224,566 (avg)	Cheaper: location reused
Tamper detection accuracy	30/30 genuine verified, 30/30 tampered detected	100% on test set
50 concurrent history requests	135.1 ms total	About 2.7 ms per request
Login (bcrypt compare + JWT)	155.0 ms	bcrypt cost factor 10

Observations: read operations (history and verify) are about 7–8 times faster than writes because they are free eth_call requests that need no mining. Gas cost is dominated by string storage, so storing only a fixed-length hash instead of the document keeps every registration well under 0.5 million gas. On a public network the write latency would rise to the block time (about 12 seconds on Ethereum), while reads stay fast.

CHAPTER 7: CONCLUSION AND FUTURE ENHANCEMENTS

7.1 Summary

The Blockchain Land Registry shows that a smart contract combined with cloud document storage gives a practical, tamper-evident land record system. Ownership records and their full history are immutable on the Ethereum ledger. Deeds are stored cheaply off-chain, and any party can check a document's authenticity in about 16 ms. The system is a complete web workflow: registrar authentication, registration, transfer, public search and public verification.

7.2 Key Achievements

- Solidity contract with duplicate protection, append-only history, events and on-chain hash verification.
- Detection of every tampered document in testing (30/30), with no false positives on genuine deeds (30/30).
- Low cost: about 0.37 M gas per registration and 0.22 M gas per transfer.
- Secure REST API with JWT, bcrypt, PDF-only uploads and server-side hashing.
- Integration of blockchain (Ethereum, Hardhat, Ganache, ethers.js) with cloud storage (Cloudinary, AWS).

7.3 Limitations

- A single server wallet signs all transactions; individual registrars are not identified on-chain.
- The development chain is local; data resets when Ganache restarts.
- Registrar accounts are kept in a JSON file, which is not suitable for production.
- Verification checks only the latest deed; older deeds must be compared with the history manually.

7.4 Future Enhancements

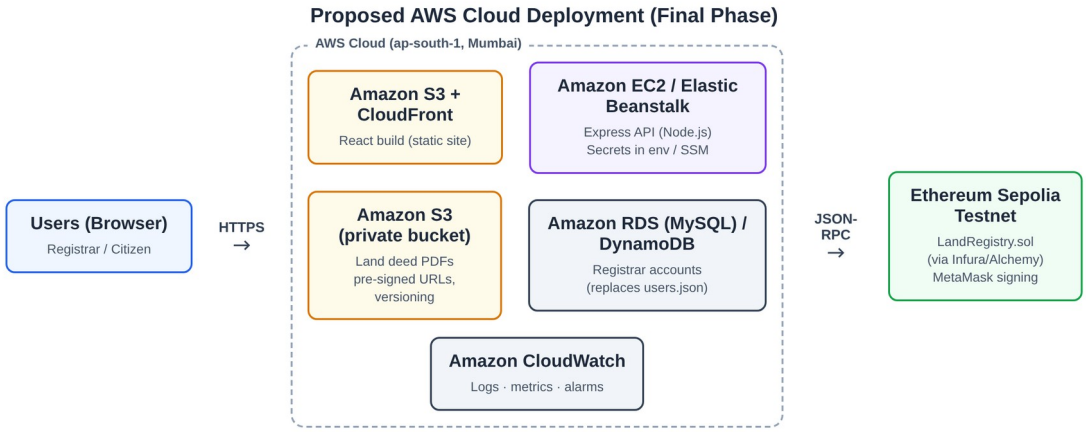


Figure 7.1: Proposed AWS deployment architecture

- **AWS deployment:** React build on S3 + CloudFront, Express API on EC2 / Elastic Beanstalk, deeds in a private S3 bucket with versioning and pre-signed URLs, logs in CloudWatch.
- **Database:** move registrar accounts to Amazon RDS (MySQL) or DynamoDB.
- **Public testnet:** deploy to Ethereum Sepolia and let users sign with MetaMask.
- **Role-based access:** Registrar, Owner and Verifier roles enforced in the contract (onlyRegistrar modifier).
- **QR code** on printed deeds linking to the on-chain record for instant verification.

REFERENCES

[1] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008. [Online]. Available: https://bitcoin.org/bitcoin.pdf

[2] V. Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform," Ethereum White Paper, 2014.

[3] G. Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger," Ethereum Yellow Paper, 2014.

[4] V. Thakur, M. N. Doja, Y. K. Dwivedi, T. Ahmad and G. Khadanga, "Land records on Blockchain for implementation of Land Titling in India," International Journal of Information Management, vol. 52, 101940, 2020.

[5] S. Krishnapriya and G. Sarath, "Securing Land Registration using Blockchain," Procedia Computer Science, vol. 171, pp. 1708–1715, 2020.

[6] M. Shuaib, S. M. Daud, S. Alam and W. Z. Khan, "Blockchain-based framework for secure and reliable land registry system," TELKOMNIKA, vol. 18, no. 5, pp. 2560–2571, 2020.

[7] Lantmäteriet, Telia, ChromaWay and Kairos Future, "The Land Registry in the Blockchain – Testbed," 2017.

[8] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," FIPS PUB 180-4, 2015.

[9] J. Benet, "IPFS – Content Addressed, Versioned, P2P File System," arXiv:1407.3561, 2014.

[10] M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, 2015.

[11] Solidity Documentation, https://docs.soliditylang.org

[12] Hardhat Documentation, https://hardhat.org/docs

[13] ethers.js v6 Documentation, https://docs.ethers.org/v6/

[14] Cloudinary Upload API Reference, https://cloudinary.com/documentation/image_upload_api_reference

[15] Amazon S3 User Guide, https://docs.aws.amazon.com/s3/